

CrewAnywhere2.0 — Master Architecture V4 (Long Version) - Part 1

Based on the canonical baseline structure from the uploaded short version document

Part	Coverage
Part 1	Platform Foundation + Event Layer
Part 2	Marketplace + Hiring Layer
Part 3	Workforce Operations Layer
Part 4	Payments + Governance + Canonical Systems

1. Document Information

Field	Value
Product Name	CrewAnywhere2.0
Document Type	Master Architecture (Long Version)
Document Version	V4
Document Part	Part 1
Architecture Status	Canonical Approved Baseline
Platform Type	Marketplace + Workforce Operations Platform
Primary Scope	Marketplace, Staffing, Workforce Operations, Operational Execution
Architecture Standard	Combined Lean FDR + Developer-Ready Technical FDR

Canonical References	Master Database Architecture Updated V3 + Master Saved Memory Architecture
Target Audience	Product Team, UI/UX Team, Backend Engineers, Frontend Engineers, DevOps, Operations Team
Architecture Philosophy	Lean, Modular, Status-Driven, Scalable
Primary Objective	Establish the canonical operational architecture baseline for CrewAnywhere2.0
Database Philosophy	Event-Centric Workforce Operations Architecture
Operational Model	Marketplace → Hiring → Assignment → Shift → Execution
Canonical Source of Truth	CrewAnywhere2.0 Architecture Documents

2. Platform Positioning

CrewAnywhere2.0 is positioned as a:

- Marketplace + Workforce Operations Platform
- Event Workforce Operating Infrastructure
- Staffing + Operational Execution Ecosystem
- Workforce Coordination Platform
- Event Staffing Infrastructure Layer

The platform combines multiple operational domains into a unified architecture:

Domain	Purpose
Talent Marketplace	Workforce discovery
Hiring Infrastructure	Proposal and hiring workflows
Assignment Management	Workforce engagement orchestration
Shift Operations	Attendance and operational execution
Event Operations	Real-world workforce coordination
Financial Infrastructure	Escrow, payout, withdrawal

Compliance Infrastructure KYB, KYC, tax validation

AI Assistance Layer Staffing optimization and recommendations

Core Platform Vision

CrewAnywhere2.0 is not designed as a simple job board.

The platform is designed as:

An operational workforce infrastructure platform capable of managing the entire staffing lifecycle from workforce discovery to operational execution and financial settlement.

This includes:

- Marketplace discovery
 - Hiring workflows
 - Workforce assignment orchestration
 - Event-day attendance operations
 - Shift execution
 - Financial processing
 - Operational tracking
 - Workforce coordination
-

Marketplace + Operations Hybrid Architecture

Traditional platforms often separate:

- Job marketplace systems
- Operational workforce systems
- Attendance systems
- Financial systems

CrewAnywhere2.0 unifies all layers into a single operational ecosystem.

Canonical lifecycle:

Registration

→ Marketplace Discovery

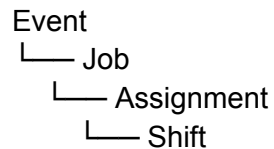
→ Proposal Submission

- Hiring
 - Assignment Creation
 - Shift Scheduling
 - Event-Day Execution
 - Payment Release
 - Withdrawal
-

3. Operational Hierarchy

CrewAnywhere2.0 follows a strict operational hierarchy.

Canonical hierarchy:



3.1 Event Layer

The Event layer represents the highest operational container.

Responsibilities

- Operational container
- Workforce demand grouping
- Multi-role staffing coordination
- Schedule ownership
- Location ownership
- Operational oversight

Examples

- Music Festival
- Conference
- Sporting Event
- Corporate Activation
- Exhibition

- Production Event
-

3.2 Job Layer

Jobs exist under Events.

A single Event may contain multiple Jobs.

Examples

Event	Jobs
Festival	Security, Registration, Camera Operator
Conference	Hostess, Stage Crew, Audio Engineer
Production	Lighting Crew, Production Assistant

Job Responsibilities

- Staffing requirement definition
 - Role requirement management
 - Proposal intake
 - Hiring workflows
 - Staffing fulfillment tracking
-

3.3 Proposal Layer

Proposal is the canonical hiring lifecycle object.

Proposal responsibilities:

- Workforce application lifecycle
- Hiring communication
- Negotiation workflow
- Proposal terms management
- Hiring decision tracking

Important Architecture Rule

Proposal is NOT an operational execution object.

Proposal responsibilities end after:

Hired

After hiring:

Assignment becomes authoritative

3.4 Assignment Layer

Assignment represents:

Official workforce engagement.

Assignments are created only after:

- Proposal finalized
 - Hire confirmed
 - Payment funded
 - Operational engagement approved
-

Assignment Responsibilities

- Workforce engagement
 - Operational scheduling
 - Shift orchestration
 - Crew allocation
 - Operational tracking
-

3.5 Shift Layer

Shift is the operational attendance object.

Shift responsibilities:

- Attendance verification
 - Check-in/check-out
 - Operational execution tracking
 - Workforce attendance monitoring
 - Event-day execution
-

3.6 Payment Layer

Payment architecture is separated from operational execution.

Responsibilities:

- Escrow management
 - Milestone funding
 - Release handling
 - Refund handling
 - Withdrawal processing
-

3.7 Architectural Responsibility Separation

Layer	Responsibility
Event	Macro operational container
Job	Staffing lifecycle
Proposal	Hiring lifecycle
Assignment	Workforce engagement lifecycle
Shift	Daily operational execution lifecycle
Payment	Financial lifecycle

4. Architecture Principles

CrewAnywhere2.0 follows strict architecture principles.

4.1 Proposal-Centric Hiring Architecture

Hiring is proposal-driven.

The Proposal object remains the canonical hiring lifecycle object.

This architecture enables:

- Clean hiring separation
- Negotiation flexibility
- Lean lifecycle management
- Scalable hiring workflows

4.2 Assignment-Based Operational Architecture

Operational execution is separated from hiring.

After hire:

Proposal → Assignment

This ensures:

- Clean operational tracking
- Workforce execution isolation
- Scalable workforce management

4.3 Shift-Based Attendance Architecture

Attendance is handled at Shift level.

Benefits:

- Accurate workforce tracking
- Shift-level operational validation
- Attendance auditing
- Multi-shift workforce support

4.4 Modular Scalable Architecture

Every operational layer is modular.

Benefits:

- Independent scaling
 - Service isolation
 - Easier backend maintenance
 - Future microservice compatibility
-

4.5 Status-Driven Architecture

CrewAnywhere2.0 is heavily status-driven.

Every layer uses canonical ENUM states.

Examples:

Layer	Status Examples
Account	Active, Suspended
Event	Draft, Open
Job	Reviewing, Filled
Proposal	Applied, Hired
Assignment	Active, Completed
Shift	Checked-In, Completed

4.6 Separation of Concerns

The platform intentionally separates:

Layer Separation	Purpose
------------------	---------

Hiring vs Operations	Cleaner lifecycle management
Assignment vs Shift	Engagement vs attendance
Marketplace vs Execution	Scalability
Payment vs Operations	Financial isolation
Compliance vs Marketplace	Regulatory separation

4.7 Developer-Ready Architecture

The architecture is intentionally backend-oriented.

Includes:

- Canonical ENUMs
 - Lifecycle consistency
 - Dependency rules
 - Validation systems
 - State transition enforcement
-

5. Platform Layers

CrewAnywhere2.0 is divided into operational platform layers.

5.1 Account Layer

Responsibilities

- Authentication
- Account lifecycle
- Access management
- Session handling
- Email verification

Core Components

- Login
 - Registration
 - Password recovery
 - OAuth
 - Session management
-

5.2 Marketplace Layer

Responsibilities

- Talent discovery
- Job browsing
- Search and filtering
- Marketplace visibility
- Saved jobs

Features

- Search engine
 - Best match sorting
 - Recommendation systems
 - Marketplace indexing
-

5.3 Hiring Layer

Responsibilities

- Proposal lifecycle
 - Hiring workflows
 - Negotiation
 - Candidate review
 - Hiring decisions
-

5.4 Assignment Layer

Responsibilities

- Workforce engagement
 - Assignment scheduling
 - Operational coordination
 - Assignment lifecycle management
-

5.5 Shift Layer

Responsibilities

- Attendance tracking
 - Event-day execution
 - Shift verification
 - Workforce attendance
-

5.6 Payment Layer

Responsibilities

- Escrow funding
 - Payment release
 - Refunds
 - Withdrawal
 - Financial reconciliation
-

5.7 Compliance Layer

Responsibilities

- KYB
 - KYC
 - Tax compliance
 - Verification systems
-

5.8 AI Assistance Layer

Responsibilities

- Skills recommendations
 - Budget suggestions
 - Role suggestions
 - Staffing optimization assistance
-

6. Account Status Architecture

Account status architecture controls authentication lifecycle and access permissions.

6.1 account_status ENUM

PENDING_VERIFICATION
ACTIVE
SUSPENDED
DELETED

6.2 Status Definitions

Status	Meaning
PENDING_VERIFICATION	Email verification incomplete
ACTIVE	Fully active account
SUSPENDED	Restricted platform access
DELETED	Account removed or archived

6.3 Applies To

The account status system applies to:

- Business Users
 - Crew Users
 - Admin Users (future-ready)
-

6.4 Architecture Responsibilities

The account status system controls:

- Authentication access
 - Login permissions
 - Verification enforcement
 - Suspension enforcement
 - Security management
-

6.5 Validation Rules

Rule	Description
Email must be verified	Required before ACTIVE
Suspended accounts blocked	Login restriction
Deleted accounts inaccessible	Permanent restriction

6.6 Canonical State Transitions

PENDING_VERIFICATION
→ ACTIVE

ACTIVE
→ SUSPENDED

SUSPENDED
→ ACTIVE

ACTIVE
→ DELETED

7. Business Registration Architecture

Business registration architecture controls creation of business accounts.

7.1 Registration Flow

Canonical flow:

Select User Type
→ Choose Signup Method
→ Validation
→ Account Creation
→ Database Record Creation
→ Email Verification
→ Dashboard Routing

7.2 Supported Signup Methods

Method	Supported
Google OAuth	Yes
LinkedIn OAuth	Yes
Email Signup	Yes

7.3 Registration Data

Required Fields

Field	Required
Full Name	Yes

Email	Yes
Password	Email Signup Only
Terms Acceptance	Yes

7.4 OAuth Architecture

OAuth registration supports:

- Prefilled profile data
 - Faster onboarding
 - Reduced form friction
-

7.5 Validation Rules

Validation includes:

- Required fields
 - Email format validation
 - Duplicate email detection
 - Password strength validation
 - Terms acceptance validation
-

7.6 Email Verification Architecture

After registration:

Verification Email Sent
→ Verification Pending
→ Email Verified
→ Account Activated

7.7 Dashboard Routing Logic

After successful verification:

User Type	Dashboard
Business	Business Dashboard
Crew	Crew Dashboard

8. Business Readiness Architecture

Business readiness architecture determines whether a business can operate inside the marketplace.

8.1 business_ready Logic

Business Ready requires:

Company Profile Completed
AND
Billing & Payment Setup Completed
AND
Tax Setup Completed
AND
KYB Submitted

8.2 verified_business Logic

Verified Business requires:

KYB Status = APPROVED

8.3 Architecture Separation

Layer	Purpose
Business Ready	Marketplace operational readiness
Verified Business	Compliance & trust readiness

8.4 Business Ready Enables

Feature	Requires Business Ready
Publish Jobs	Yes
Receive Applications	Yes
Manage Draft Jobs	Yes
View Applicants	Yes

8.5 Verified Business Enables

Feature	Requires Verified Business
Hire Crew	Yes
Payment Processing	Yes
Contract Operations	Yes
Financial Operations	Yes

8.6 KYB Lifecycle

Canonical KYB statuses:

NOT_STARTED
 IN_PROGRESS
 SUBMITTED
 PENDING_REVIEW

APPROVED
REJECTED

8.7 Architecture Principle

Important architecture rule:

Business Ready $\neq$ Verified Business

This separation allows:

- Faster marketplace onboarding
 - Operational scalability
 - Compliance flexibility
-

9. Event & Job Creation Architecture

Event and Job architecture forms the operational foundation of CrewAnywhere2.0.

9.1 Canonical Entity Relationship

Event ID = Parent Entity

Job ID = Child Entity

9.2 Event Responsibilities

Events contain:

- Operational metadata
- Event schedules
- Locations
- Staffing requirements

- Operational timelines
-

9.3 Job Responsibilities

Jobs contain:

- Role requirements
 - Skills requirements
 - Staffing quantities
 - Compensation terms
 - Proposal intake
-

9.4 Multi-Role Event Architecture

Single events may contain multiple jobs.

Example:

Event	Jobs
Music Festival	Security, Camera Crew, Registration
Conference	Hostess, Stage Manager, Audio Engineer

9.5 Event Creation Flow

Canonical flow:

Create Event
→ Enter Event Details
→ Save Draft
→ Create Jobs
→ Validate Staffing Requirements
→ Publish Event

9.6 Job Creation Flow

Canonical flow:

Create Job

→ Define Role

→ Define Skills

→ Define Staffing Count

→ Define Budget

→ Save Job

9.7 AI-Assisted Job Creation

AI assistance supports:

- Role suggestions
 - Skills recommendations
 - Budget estimation
 - Requirement recommendations
-

9.8 AI Architecture Limitation

Important architecture rule:

AI assists only.

User remains authoritative editor.

AI cannot:

- Auto-publish events
 - Auto-hire crew
 - Override operational data
-

9.9 Validation Rules

Validation

Requirement

Event must contain jobs	Required
job_count >= 1	Mandatory
Required job fields completed	Mandatory
Draft validation before publish	Required

9.10 Publish Readiness Logic

Event publish requires:

- Event details completed
 - At least one valid Job
 - Required validation passed
-

10. Event Status Architecture

Event status architecture controls macro operational lifecycle.

10.1 Event Status ENUM

Draft
Open
Closed

10.2 Status Definitions

Status	Meaning
Draft	Event being prepared
Open	Event operationally active

Close Event lifecycle finalized
d

10.3 Status Responsibilities

Status	Responsibilities
Draft	Editable setup phase
Open	Active staffing & operations
Close	Finalized operational lifecycle

d

10.4 State Transition Logic

Canonical flow:

Draft
→ Open
→ Closed

10.5 Draft State Rules

Draft allows:

- Event editing
- Job editing
- Staffing setup
- Operational preparation

Draft blocks:

- Marketplace visibility
 - Proposal intake
-

10.6 Open State Rules

Open enables:

- Marketplace publishing
 - Proposal submissions
 - Hiring workflows
 - Operational staffing
-

10.7 Closed State Rules

Closed disables:

- New proposals
- New hiring
- Operational modifications

Closed allows:

- Historical reporting
 - Audit tracking
 - Payment reconciliation
-

10.8 Event ↔ Job Dependency Rules

Event Status	Allowed Job Status
Draft	Draft
Open	Open, Reviewing, Filled, Active
Closed	Completed, Closed, Expired

10.9 Architecture Principle

Important architecture rule:

Event controls macro operational lifecycle.

Jobs control staffing lifecycle.

This separation enables:

- Flexible staffing
- Independent job fulfillment
- Scalable workforce operations

CrewAnywhere2.0 — Master Architecture V4 (Long Version) - Part 2

Based on the canonical baseline structure from the uploaded short version document

11. Job Status Architecture

Job status architecture governs the staffing fulfillment lifecycle inside the platform.

Jobs are operational staffing units created under Events and are responsible for workforce demand management.

11.1 Job Status ENUM

Draft

Open

Reviewing

Filled

Active

Completed

Closed

Expired

11.2 Status Definitions

Status	Meaning
Draft	Job is being configured
Open	Job accepting applications
Reviewing	Business reviewing proposals
Filled	Required staffing fulfilled
Active	Workforce execution active
Completed	Operational work finished
Closed	Administrative archival state
Expired	Application period expired

11.3 Architecture Purpose

Job statuses control:

- Staffing lifecycle progression
 - Marketplace visibility
 - Proposal intake
 - Hiring state management
 - Workforce operational readiness
-

11.4 Canonical Job Lifecycle

Primary lifecycle:

Draft

→ Open

→ Reviewing

→ Filled

→ Active

→ Completed

→ Closed

Alternative lifecycle:

Draft

→ Open

→ Expired

11.5 Draft Status Rules

Draft allows:

- Editing role information
- Editing compensation

- Editing staffing count
- Editing skills requirements

Draft blocks:

- Marketplace visibility
 - Proposal submissions
 - Hiring workflows
-

11.6 Open Status Rules

Open enables:

- Marketplace publishing
 - Proposal submissions
 - Candidate discovery
 - Staffing operations
-

11.7 Reviewing Status Rules

Reviewing indicates:

- Proposal evaluation in progress
- Candidate review active
- Hiring decisions pending

Reviewing still allows:

- Additional applications
 - Candidate communication
 - Proposal negotiation
-

11.8 Filled Status Rules

Filled means:

Required staffing quantity reached

Filled blocks:

- Additional hiring
- New applications

Filled enables:

- Workforce scheduling
 - Operational preparation
-

11.9 Active Status Rules

Active indicates:

Operational workforce execution has started

Triggered when:

- Assignments become operationally active
 - Shift execution begins
 - Event-day operations start
-

11.10 Completed Status Rules

Completed indicates:

- Operational work finished
 - Required assignments completed
 - Workforce execution successfully finalized
-

11.11 Closed Status Rules

Closed represents:

- Administrative archival state
- Historical record finalization
- Permanently closed operational lifecycle

Closed disables:

- Editing
 - Hiring
 - Staffing modifications
-

11.12 Expired Status Rules

Expired indicates:

- Application deadline exceeded
- Staffing opportunity no longer available

Expired may occur because of:

- Event schedule changes
 - Staffing cancellation
 - Unfulfilled timeline expiration
-

11.13 Architecture Principle

Important architecture rule:

Job lifecycle controls staffing fulfillment.

Operational execution occurs separately.

12. Event ↔ Job Dependency

Event and Job architecture follows strict dependency enforcement.

12.1 Dependency Philosophy

Events govern:

- Macro operational lifecycle

Jobs govern:

- Staffing lifecycle

This separation enables scalable workforce coordination.

12.2 Canonical Relationship

events.event_id

↓

jobs.event_id

12.3 Dependency Matrix

Event Status	Allowed Job Status
Draft	Draft
Open	Open, Reviewing, Filled, Active
Closed	Completed, Closed, Expired

12.4 Draft Event Rules

When Event = Draft:

Allowed Job Status:

Draft

Blocked:

- Open
- Reviewing
- Active

Purpose:

Prevent workforce operations before event publication.

12.5 Open Event Rules

When Event = Open:

Allowed Job Status:

Open

Reviewing

Filled

Active

Purpose:

Allow active staffing and operational workflows.

12.6 Closed Event Rules

When Event = Closed:

Allowed Job Status:

Completed

Closed

Expired

Purpose:

Ensure lifecycle consistency and operational closure.

12.7 Validation Rules

Validation	Requirement
Event must exist before Job creation	Required
Jobs inherit Event lifecycle boundaries	Enforced
Closed Events cannot reopen Jobs	Restricted
Event deletion impacts Jobs	Cascading dependency

12.8 Architecture Principle

Important architecture rule:

Event is the operational parent entity.

Job is the staffing child entity.

13. Crew Registration Architecture

Crew registration architecture governs workforce account onboarding.

13.1 Canonical Registration Flow

Select User Type

→ Sign Up as Crew

→ Choose Signup Method

→ Validation

→ Account Creation

→ Database Record Creation

→ Email Verification

→ Start Profile Creation

13.2 Supported Signup Methods

Method	Supported
--------	-----------

Google OAuth	Yes
--------------	-----

LinkedIn OAuth	Yes
----------------	-----

Email Signup	Yes
--------------	-----

13.3 Registration Objectives

The registration system is designed to:

- Reduce onboarding friction
 - Increase marketplace conversion
 - Enable scalable workforce onboarding
 - Support global workforce growth
-

13.4 Registration Fields

Required Fields

Field	Required
Full Name	Yes
Email	Yes
Password	Email Signup Only

Terms Yes
Acceptance

Optional:

- Country
 - Referral information
-

13.5 OAuth Architecture

OAuth integrations support:

- Faster onboarding
 - Prefilled profile information
 - Reduced manual data entry
 - Improved user conversion
-

13.6 Validation Rules

Validation includes:

- Required field validation
 - Email format validation
 - Password strength validation
 - Password confirmation validation
 - Duplicate email validation
-

13.7 Email Verification Architecture

Canonical flow:

Registration Completed

→ Verification Email Sent

→ Email Verified

→ Account Activated

13.8 Pending Verification State

Before verification:

Blocked:

- Marketplace participation
 - Profile publishing
 - Proposal submissions
-

13.9 Architecture Principle

Important architecture rule:

Registration creates platform identity.

Profile creation creates marketplace identity.

14. Crew Profile Creation Architecture

Profile creation architecture governs workforce marketplace identity creation.

14.1 Supported Profile Sources

Profiles may originate from:

Source	Supported
LinkedIn Import	Yes
Manual Form	Yes
CV Upload	Yes

14.2 Canonical Profile Creation Flow

Profile Creation

- Basic Information
- Skills & Preferences
- Professional Details
- Validation
- Preview Profile
- Publish Profile

14.3 Basic Information Section

Includes:

- Full Name
- Gender
- Date of Birth

- Profile Image
 - Job Title
 - Introduction
 - Video Introduction (optional)
-

14.4 Job Preferences & Skills Section

Includes:

- Job Categories
 - Skill Tags
 - Preferred Locations
 - Availability
-

14.5 Professional Details Section

Includes:

- Experience
 - Certifications
 - Expected Rates
 - Industry
 - Languages
-

14.6 Validation Rules

Validation includes:

- Required profile fields
 - Completeness validation
 - Skills validation
 - Media validation
-

14.7 Profile Completion Logic

Incomplete profiles:

- Cannot publish
 - Cannot appear in marketplace
 - Cannot submit proposals
-

14.8 Publish Profile Flow

Canonical flow:

Complete Profile

→ Validation Passed

→ Preview Profile

→ Publish Profile

→ Marketplace Visibility Enabled

14.9 Architecture Principle

Important architecture rule:

Marketplace identity is profile-driven, not account-driven.

15. Marketplace Readiness Architecture

Marketplace readiness architecture determines workforce eligibility inside the marketplace.

15.1 marketplace_ready Logic

Marketplace Ready requires:

Email Verified

AND

Required Profile Fields Completed

AND

Profile Published

AND

Minimum Profile Score Achieved

15.2 Marketplace Ready Enables

Feature	Requires Marketplace Ready
Marketplace Visibility	Yes
Job Applications	Yes
Proposal Submission	Yes
Hiring Eligibility	Yes

15.3 Marketplace Restrictions

Without Marketplace Ready:

Blocked:

- Marketplace visibility
 - Proposal submission
 - Hiring participation
-

15.4 Profile Score Logic

Profile scoring may consider:

- Profile completeness
 - Skills completion
 - Certifications
 - Verification status
 - Media uploads
-

15.5 Marketplace Visibility Rules

Only Marketplace Ready users appear in:

- Marketplace search
 - Candidate discovery
 - Recommendation systems
-

15.6 Architecture Principle

Important architecture rule:

Marketplace participation is readiness-driven, not registration-driven.

16. Identity Verification Architecture

Identity verification architecture supports workforce trust and compliance infrastructure.

16.1 Supported Verification Documents

Document Type	Supported
Passport	Yes
National ID	Yes
Driver License	Yes
Government ID	Yes

16.2 Supported Verification Methods

Verification methods include:

- Selfie verification
 - Video verification
 - Phone verification
 - Proof of address
-

16.3 Verification Lifecycle

Canonical statuses:

SUBMITTED

APPROVED

ADDITIONAL_INFO_REQUESTED

REJECTED

16.4 Verification Objectives

Identity verification supports:

- Marketplace trust
- Fraud prevention
- Workforce validation
- Regulatory compliance

16.5 Verification Dependencies

Verification may be required for:

- High-value assignments
- Financial operations
- Withdrawal processing
- Platform trust enforcement

16.6 Additional Information Flow

Canonical flow:

Verification Submitted

→ Additional Information Requested

→ User Resubmits Information

→ Verification Review Continues

16.7 Architecture Principle

Important architecture rule:

Identity verification is a trust layer, not an onboarding layer.

17. Marketplace & Job Discovery Architecture

Marketplace discovery architecture governs workforce-job matching.

17.1 Marketplace Capabilities

Marketplace supports:

- Search-based discovery
 - Filtering systems
 - Recommendation systems
 - Sorting systems
-

17.2 Search Features

Search supports:

- Keyword matching
 - Skill-based matching
 - Category filtering
 - Location filtering
-

17.3 Sorting Systems

Supported sorting includes:

- Best Match
 - Most Recent
 - Recommended
 - Availability-based
-

17.4 Marketplace Separation Logic

Canonical separation:

Layer	Purpose
Saved Jobs	Bookmarking
Applications	Proposal lifecycle
Current Jobs	Operational engagement

17.5 Current Job Tracking

Current Jobs includes:

Applied

In Review

Hired

Completed

17.6 Marketplace Discovery Flow

Canonical flow:

Search Jobs

→ Filter Results

→ View Job Details

→ Save OR Apply

→ Submit Proposal

17.7 Job Detail Responsibilities

Job Detail pages contain:

- Event information
 - Role information
 - Compensation details
 - Requirements
 - Business information
-

17.8 Architecture Principle

Important architecture rule:

Saved Jobs remain separate from operational hiring lifecycle.

18. Proposal Lifecycle Architecture

Proposal architecture governs hiring workflows.

18.1 Proposal Status ENUM

Applied

Offer Sent

Offer Accepted

Declined

Withdrawn

Hired

18.2 Status Definitions

Status	Meaning
Applied	Proposal submitted
Offer Sent	Business sent finalized terms
Offer Accepted	Crew accepted proposal terms
Declined	Proposal rejected

Withdrawn Crew voluntarily withdrew

Hired Operational engagement officially confirmed

18.3 Canonical Proposal Lifecycle

Primary lifecycle:

Applied

→ Offer Sent

→ Offer Accepted

→ Hired

Alternative paths:

Applied

→ Declined

Applied

→ Withdrawn

18.4 Important Architecture Rule

Critical rule:

Offer Accepted $\neq$ Hired

Hired occurs only after:

- Payment funded
 - Workforce engagement confirmed
 - Assignment creation completed
-

18.5 Proposal Responsibilities

Proposal manages:

- Hiring lifecycle
 - Candidate evaluation
 - Hiring communication
 - Negotiation workflows
 - Hiring decisions
-

18.6 Proposal Lifecycle Boundary

Proposal lifecycle ends at:

Hired

After Hired:

Operational lifecycle moves into Assignment architecture

18.7 Architecture Principle

Important architecture rule:

Proposal is a hiring object, not an operational execution object.

19. Proposal Negotiation Architecture

Proposal negotiation architecture governs hiring communication and terms discussion.

19.1 Negotiation Philosophy

Negotiation occurs through:

- Proposal Terms
- Chat Layer

NOT through:

- Additional proposal statuses
-

19.2 Proposal Terms Architecture

Proposal Terms are embedded inside the Proposal lifecycle

and are not treated as standalone Offer Objects.

The platform follows a proposal-centric hiring architecture

where hiring negotiation occurs through:

- Proposal Terms
- Chat Layer
- Proposal lifecycle transitions

NOT through separate Offer Object systems.

19.3 Canonical Negotiation Flow

Business Sends Terms

→ Crew Reviews Terms

→ Accept OR Request Changes OR Decline

19.4 Request Changes Flow

If Crew requests revisions:

Proposal Remains Active

→ Business Revises Terms

→ Crew Reviews Again

19.5 Decline Flow

If negotiations fail:

Proposal Status

→ Declined

19.6 Architecture Restrictions

The architecture intentionally avoids statuses such as:

- Negotiating
- Counter Offered

- Awaiting Response
- Reopened

Purpose:

Maintain lean scalable hiring architecture.

19.7 Dashboard Separation Architecture

Layer	Responsibility
Dashboard	Job-centric
Job Detail	Proposal-centric
Proposal Detail	Chat + Terms + Candidate Review

19.8 Architecture Principle

Important architecture rule:

Negotiation is communication-driven, not status-driven.

CrewAnywhere2.0 — Master Architecture V4 (Long Version) - Part 3

Based on the canonical baseline structure from the uploaded short version document

20. Assignment Architecture

Assignment architecture governs official workforce operational engagement inside the platform.

Assignments act as the operational bridge between hiring and workforce execution.

20.1 Assignment Philosophy

Assignment represents:

Official workforce engagement after hiring has been finalized.

Assignments separate:

- Hiring lifecycle
- Operational execution lifecycle

This separation is a core architectural principle of CrewAnywhere2.0.

20.2 Assignment Creation Requirements

Assignments are created only after:

Proposal Status = HIRED

AND

Payment Funded

AND

Operational Engagement Confirmed

20.3 Assignment Responsibilities

Assignments manage:

- Workforce engagement

- Operational scheduling
 - Workforce allocation
 - Shift orchestration
 - Workforce coordination
 - Operational tracking
-

20.4 Canonical Relationship

jobs.job_id
↓
assignments.job_id

20.5 Assignment Data Structure

Canonical Assignment object includes:

Field	Purpose
assignment_id	Unique assignment identifier
event_id	Parent event
job_id	Parent job
proposal_id	Hiring reference
crew_user_id	Assigned crew member
business_user_id	Hiring business
assignment_status	Operational lifecycle
start_date	Assignment start
end_date	Assignment end
timezone	Operational timezone
compensation_terms	Payment structure
shift_group_id	Shift relationship

20.6 Assignment Operational Role

Assignments act as:

- Workforce operational container
 - Parent object for shifts
 - Operational coordination layer
 - Workforce execution bridge
-

20.7 Assignment Creation Flow

Canonical flow:

Proposal Finalized

→ Payment Funded

→ Assignment Created

→ Assignment Scheduled

→ Shift Generation Starts

20.8 Assignment Lifecycle Philosophy

Important architecture rule:

Assignment governs engagement lifecycle.

Shift governs attendance lifecycle.

20.9 Assignment Architecture Benefits

The assignment layer enables:

- Clean workforce orchestration
- Multi-shift workforce support
- Operational scalability
- Workforce tracking
- Separation between hiring and attendance

20.10 Architecture Principle

Important architecture rule:

Assignment is an operational engagement object, not a hiring object.

21. Assignment Status Architecture

Assignment status architecture governs workforce engagement lifecycle progression.

21.1 Assignment Status ENUM

Scheduled

Active

Completed

Cancelled

21.2 Status Definitions

Status	Meaning
Scheduled	Assignment created and awaiting execution
Active	Workforce operationally active
Completed	Workforce engagement completed
Cancelled	Operational engagement cancelled

21.3 Canonical Assignment Lifecycle

Primary lifecycle:

Scheduled
→ Active
→ Completed

Alternative lifecycle:

Scheduled
→ Cancelled

21.4 Scheduled Status Rules

Scheduled indicates:

- Assignment officially created
- Workforce engagement confirmed
- Shift generation prepared
- Operational execution not yet started

21.5 Active Status Rules

Active indicates:

Operational workforce execution has started

Typically triggered when:

- Shift checked-in
- Shift in progress
- Operational attendance verified

21.6 Completed Status Rules

Completed indicates:

- All required shifts completed
- Workforce execution finalized

- Operational engagement finished successfully
-

21.7 Cancelled Status Rules

Cancelled indicates:

- Assignment terminated before completion
 - Event cancellation
 - Staffing cancellation
 - Operational interruption
-

21.8 Trigger Logic

Assignment Status	Trigger
Scheduled	Business hires crew
Active	Shift checked-in or in progress
Completed	All shifts completed
Cancelled	Event/business cancellation

21.9 Architecture Principle

Important architecture rule:

Assignment lifecycle tracks workforce engagement, not attendance.

22. Job ↔ Assignment Dependency

Job and Assignment architecture follows strict dependency enforcement.

22.1 Dependency Philosophy

Jobs manage:

- Staffing fulfillment lifecycle

Assignments manage:

- Workforce operational engagement
-

22.2 Canonical Relationship

jobs.job_id

↓

assignments.job_id

22.3 Dependency Matrix

Job Status	Assignment Behavior
Draft	No assignments allowed
Open	No assignments allowed
Reviewing	No assignments allowed
Filled	Assignment scheduling enabled
Active	Assignment active
Completed	Assignment completed
Closed	Assignment cancelled/finalized
Expired	None or cancelled

22.4 Draft/Open/Reviewing Rules

When Job is:

- Draft
- Open
- Reviewing

Assignments are blocked because:

Hiring process not finalized

22.5 Filled Status Dependency

Filled enables:

- Assignment creation
 - Workforce scheduling
 - Shift preparation
-

22.6 Active Status Dependency

When Job = Active:

Assignments may become:

Active

This indicates operational execution has begun.

22.7 Completed Status Dependency

When Job = Completed:

Assignments should also transition into:

Completed

Purpose:

Maintain lifecycle consistency.

22.8 Architecture Principle

Important architecture rule:

Assignments cannot exist without finalized staffing fulfillment.

23. Shift Architecture

Shift architecture governs operational attendance execution.

Shifts are generated from Assignments and represent actual operational work sessions.

23.1 Canonical Relationship

assignments.assignment_id

↓

shifts.assignment_id

23.2 Shift Responsibilities

Shifts manage:

- Attendance verification
 - Check-in/check-out
 - Workforce execution
 - Operational attendance tracking
 - Event-day workforce monitoring
-

23.3 Shift Generation Logic

Shifts are generated from:

Assignment Creation

Assignments may generate:

- Single shifts
- Multi-day shifts
- Repeating shifts
- Rotational schedules

23.4 Shift Data Structure

Canonical Shift object includes:

Field	Purpose
shift_id	Unique shift identifier
assignment_id	Parent assignment
crew_user_id	Assigned crew
shift_status	Attendance lifecycle
start_time	Shift start
end_time	Shift end
check_in_time	Attendance timestamp
check_out_time	Completion timestamp
supervisor_id	Operational supervisor
attendance_validation_method	Verification source

23.5 Shift Operational Role

Shift acts as:

- Attendance object
 - Operational execution object
 - Workforce attendance tracker
 - Payment release trigger source
-

23.6 Architecture Principle

Important architecture rule:

Shift represents attendance execution, not workforce engagement ownership.

24. Shift Status Architecture

Shift status architecture governs workforce attendance lifecycle.

24.1 Shift Status ENUM

Scheduled
Checked-In
In Progress
Completed
No-Show
Cancelled

24.2 Status Definitions

Status	Meaning
Scheduled	Shift awaiting execution

Checked-In	Attendance verified
In Progress	Workforce actively working
Completed	Shift execution finished
No-Show	Crew failed to attend
Cancelled	Shift cancelled

24.3 Canonical Shift Lifecycle

Primary lifecycle:

Scheduled
→ Checked-In
→ In Progress
→ Completed

Alternative lifecycle:

Scheduled
→ No-Show

Alternative cancellation lifecycle:

Scheduled
→ Cancelled

24.4 Checked-In Rules

Checked-In indicates:

- Attendance verified
- Workforce arrived
- Operational execution ready

Verification methods may include:

- QR verification

- GPS validation
 - Supervisor approval
-

24.5 In Progress Rules

In Progress indicates:

Workforce operational execution active

24.6 Completed Rules

Completed indicates:

- Shift execution finalized
 - Check-out verified
 - Operational attendance completed
-

24.7 No-Show Rules

No-Show indicates:

- Attendance window expired
 - Crew failed to check in
 - Operational attendance failure
-

24.8 Trigger Examples

Shift Status	Trigger
Checked-In	QR verification
In Progress	Successful attendance start
Completed	Check-out verification

No-Show Attendance deadline
 exceeded

24.9 Architecture Principle

Important architecture rule:

Shift lifecycle governs attendance execution only.

25. Assignment ↔ Shift Dependency Architecture

Assignment and Shift architecture follows strict operational dependency enforcement.

25.1 Dependency Philosophy

Assignments manage:

- Workforce engagement lifecycle

Shifts manage:

- Daily operational attendance lifecycle
-

25.2 Dependency Matrix

Assignment Status	Allowed Shift Status
Scheduled	Scheduled
Active	Checked-In, In Progress

Completed

Completed

Cancelled

Cancelled, No-Show

25.3 Scheduled Assignment Rules

When Assignment = Scheduled:

Allowed Shift Status:

Scheduled

Purpose:

Prevent operational execution before workforce activation.

25.4 Active Assignment Rules

When Assignment = Active:

Allowed Shift Status:

Checked-In

In Progress

Purpose:

Allow operational workforce execution.

25.5 Completed Assignment Rules

When Assignment = Completed:

Allowed Shift Status:

Completed

Purpose:

Maintain operational completion consistency.

25.6 Cancelled Assignment Rules

When Assignment = Cancelled:

Allowed Shift Status:

Cancelled

No-Show

Purpose:

Ensure attendance closure consistency.

25.7 Architecture Principle

Important architecture rule:

Assignments govern engagement state.

Shifts govern attendance state.

26. Event-Day Operational Execution Architecture

Event-Day Operational Execution architecture governs real-world workforce attendance and operational workflows.

26.1 Operational Philosophy

This layer transforms:

Digital workforce assignments

→

Real-world workforce execution

26.2 Canonical Operational Flow

Shift Scheduled

→ Crew Check-In

→ QR Verification

→ In Progress

→ Check-Out

→ Supervisor Confirmation

→ Completed

26.3 Check-In Architecture

Check-in supports:

- QR attendance scanning
 - GPS validation
 - Supervisor approval
 - Manual override
-

26.4 Check-In Flow

Canonical flow:

Crew Opens Shift

→ Scan QR Code

→ Verification Successful

→ Shift Status = Checked-In

Fallback flow:

QR Failure

- Supervisor Manual Approval
 - Check-In Approved
-

26.5 Check-Out Architecture

Check-out validates:

- Work completion
 - Operational attendance
 - Hours worked
 - Supervisor approval
-

26.6 Check-Out Flow

Canonical flow:

Shift Ends

- Crew Check-Out
 - Supervisor Confirms
 - Shift Completed
 - Payment Trigger Triggered
-

26.7 Operational Validation Methods

Supported methods:

Validation Method	Purpose
QR Verification	Attendance validation
GPS Validation	Location verification
Supervisor Approval	Operational override
Manual Override	Fallback operations

26.8 Operational Objectives

The operational execution layer supports:

- Workforce tracking
 - Attendance validation
 - Operational coordination
 - Payment release triggering
 - Real-world execution monitoring
-

26.9 Architecture Principle

Important architecture rule:

Event-day execution is attendance-driven and operationally validated.

CrewAnywhere2.0 — Master Architecture V4 (Long Version) - Part 4

Based on the canonical baseline structure from the uploaded short version document

27. Payment Architecture

Payment architecture governs escrow funding, payment release, payout processing, and workforce financial operations.

The payment layer is intentionally separated from hiring and operational execution architecture.

27.1 Payment Philosophy

CrewAnywhere2.0 follows a controlled operational payment lifecycle.

Core principles:

- Escrow-first architecture
 - Milestone-based release
 - Operational validation before payout
 - Workforce protection
 - Business payment protection
-

27.2 Canonical Payment Lifecycle

Milestone Funded

→ Work Submitted

→ Client Approval

→ Security Hold

→ Funds Available

→ Withdrawal

27.3 Payment Responsibilities

The payment layer manages:

- Escrow funding
 - Milestone holding
 - Payment release
 - Refund processing
 - Withdrawal processing
 - Financial tracking
-

27.4 Payment Funding Architecture

Funding occurs after:

Proposal Finalized

AND

Hire Confirmed

Funding is required before:

Assignment Creation

27.5 Payment Funding Statuses

Canonical funding statuses:

Awaiting Funding
Funded
Partially Funded
Failed Funding

27.6 Payment Release Statuses

Canonical release statuses:

Pending Release
Released
Refunded

27.7 Security Hold Architecture

Security hold exists to:

- Prevent payment fraud
 - Support dispute windows
 - Protect platform financial integrity
 - Allow operational validation
-

27.8 Refund Handling

Refund flows may occur because of:

- Assignment cancellation
- Operational failure
- Workforce no-show
- Disputes

- Funding reversal
-

27.9 Payment Trigger Dependencies

Payment release depends on:

Requirement	Required
Shift completed	Yes
Operational approval	Yes
Attendance validated	Yes
Business approval	Yes

27.10 Architecture Principle

Important architecture rule:

Payment release is operationally validated, not hiring validated.

28. Finance / Tax / Withdrawal Architecture

Finance architecture governs tax compliance, payout setup, and withdrawal operations.

28.1 Finance Philosophy

The finance layer separates:

- Tax compliance
- Identity verification
- Withdrawal management
- Financial operations

28.2 Finance Architecture Components

The finance layer includes:

Layer	Purpose
Tax Setup	Tax compliance
KYC Layer	Identity validation
Withdrawal Setup	Payout configuration
Transaction Tracking	Financial history

28.3 Tax Architecture

Tax setup supports:

- Tax identification
 - Tax forms
 - Jurisdiction handling
 - Financial compliance
-

28.4 KYC Financial Layer

KYC information may include:

Field	Purpose
Legal Name	Identity validation
Government ID	Verification
Date of Birth	Compliance
Address	Financial validation

28.5 Withdrawal Methods

Supported withdrawal methods include:

- Bank Account
- PayPal
- Wise

Future-ready for:

- Stripe Connect
 - Regional payout providers
 - Alternative payout rails
-

28.6 Withdrawal Readiness Logic

Withdrawal requires:

Finance Setup Completed

AND

Withdrawal Method Linked

AND

KYC Validation Passed

28.7 Withdrawal Lifecycle

Canonical flow:

Funds Available

→ Withdrawal Requested

→ Payout Processing

→ Withdrawal Completed

28.8 Withdrawal Tracking

The platform tracks:

- Withdrawal history
 - Processing status
 - Failed payouts
 - Transaction references
-

28.9 Architecture Principle

Important architecture rule:

Financial readiness is separated from marketplace readiness.

29. Dashboard Architecture

Dashboard architecture governs operational platform visibility and workflow organization.

29.1 Dashboard Philosophy

CrewAnywhere2.0 follows a:

Job-centric operational dashboard architecture

The dashboard is designed for:

- Staffing oversight
 - Hiring visibility
 - Operational tracking
 - Workforce coordination
-

29.2 Dashboard Hierarchy

Canonical dashboard structure:

Level	Responsibility
Level 1	Job-centric dashboard
Level 2	Proposal-centric Job Detail
Level 3	Candidate Review + Chat + Hiring

29.3 Dashboard Responsibilities

Dashboard manages:

- Staffing operations
 - Proposal visibility
 - Hiring progress
 - Assignment tracking
 - Operational workforce management
-

29.4 Dashboard Architecture Rules

The dashboard intentionally avoids:

- Proposal-centric homepage structure
- Offer-centric operational routing
- Shift-centric staffing management

Purpose:

Maintain scalable workforce operations architecture.

29.5 Dashboard Status Organization

Primary dashboard filters use Job Statuses only:

Draft
Open
Reviewing
Filled
Active

Completed
Closed
Expired

29.6 Proposal Visibility Architecture

Proposal management occurs inside:

Job Detail Layer

NOT directly at dashboard level.

29.7 Dashboard Informational Indicators

Dashboard may display:

- Proposal counts
 - Unread messages
 - Hired crew counts
 - Last activity
 - Assignment counts
-

29.8 Architecture Principle

Important architecture rule:

Dashboard architecture is operationally job-centric, not proposal-centric.

30. AI-Assisted Staffing Architecture

AI-assisted staffing architecture provides operational recommendations while preserving human control.

30.1 AI Philosophy

AI acts as:

Operational assistance layer

NOT operational authority.

30.2 AI Responsibilities

AI may assist with:

- Role creation
 - Skills recommendations
 - Budget estimation
 - Staffing suggestions
 - Requirement recommendations
-

30.3 AI Restrictions

AI does NOT control:

- Final event data
 - Hiring decisions
 - Workforce execution
 - Payment decisions
 - Operational approvals
-

30.4 AI-Assisted Event Creation

AI may assist businesses with:

- Suggested staffing structures
- Suggested crew counts

- Suggested operational roles
-

30.5 AI-Assisted Marketplace Matching

AI may support:

- Candidate recommendations
 - Best match ranking
 - Workforce suggestions
-

30.6 Human Authority Principle

Important architecture rule:

Users remain authoritative decision-makers.
AI only assists operational workflows.

31. Documentation Standard Architecture

Documentation standard architecture defines official technical documentation structure for CrewAnywhere2.0.

31.1 Official Documentation Standard

Official standard:

Combined Lean FDR + Developer-Ready Technical FDR

31.2 Documentation Objectives

The documentation standard ensures:

- Developer readiness
 - Workflow consistency
 - Backend clarity
 - Scalable architecture documentation
-

31.3 Mandatory Documentation Characteristics

All official FDR/FRD documents should include:

- Lean workflows
 - Backend-ready structures
 - ENUM definitions
 - API sections
 - Validation logic
 - State transition logic
 - Security sections
 - Acceptance criteria
-

31.4 Canonical Section Structure

Standardized structure includes:

Section	Purpose
Document Information	Metadata
Overview	Module purpose
Objectives	Functional goals
Actors	User roles
Preconditions	Required states
Process Flow	Workflow
Database Schema	Backend structure
ENUMs	Status systems
APIs	Integration structure

Validation	Rules enforcement
Security	Compliance
Acceptance Criteria	Completion validation

31.5 Documentation Philosophy

Documentation is intentionally:

- Lean
 - Scalable
 - Backend-oriented
 - Developer-readable
 - Operationally structured
-

31.6 Architecture Principle

Important architecture rule:

Documentation must remain operationally aligned with canonical architecture.

32. Database Separation Architecture

Database separation architecture defines CrewAnywhere2.0 as an independent operational platform architecture.

32.1 Canonical Separation Rule

CrewAnywhere2.0 is officially treated as:

Independent architecture baseline

32.2 Separation Scope

Separated from:

- Legacy CrewAnywhere database
 - Legacy workflow structures
 - Legacy lifecycle systems
 - Legacy status architecture
-

32.3 Separation Objectives

The separation enables:

- Clean architecture evolution
 - Scalable operational design
 - Modern workforce infrastructure
 - Lean backend architecture
-

32.4 Database Philosophy

CrewAnywhere2.0 uses:

Event-centric operational database architecture

32.5 Operational Separation Principles

The platform separates:

Layer	Responsibility
Marketplace	Discovery
Proposal	Hiring
Assignment	Workforce engagement

Shift	Attendance
Payment	Financial operations

32.6 Architecture Principle

Important architecture rule:

CrewAnywhere2.0 is not an extension of legacy CrewAnywhere architecture.

33. Final Canonical Status Matrix

CrewAnywhere2.0 uses canonical lifecycle-driven status systems.

33.1 Canonical Status Matrix

Layer	Statuses
Account	Pending Verification, Active, Suspended, Deleted
Event	Draft, Open, Closed
Job	Draft, Open, Reviewing, Filled, Active, Completed, Closed, Expired
Proposal	Applied, Offer Sent, Offer Accepted, Declined, Withdrawn, Hired
Assignment	Scheduled, Active, Completed, Cancelled
Shift	Scheduled, Checked-In, In Progress, Completed, No-Show, Cancelled

33.2 Status Architecture Philosophy

Status systems enforce:

- Operational consistency

- Validation logic
 - Lifecycle control
 - Backend enforcement
 - Workflow predictability
-

33.3 Architecture Principle

Important architecture rule:

Every operational layer owns its own lifecycle system.

34. Final Operational Lifecycle

CrewAnywhere2.0 follows a unified operational workforce lifecycle.

34.1 Canonical Operational Lifecycle

Registration

- Profile Creation
 - Marketplace Readiness
 - Event Creation
 - Job Creation
 - Marketplace Discovery
 - Proposal Submission
 - Negotiation
 - Hire
 - Assignment Creation
 - Shift Generation
 - Event-Day Execution
 - Payment Release
 - Withdrawal
-

34.2 Lifecycle Philosophy

The lifecycle intentionally separates:

Phase	Responsibility
Marketplace	Discovery & Hiring
Operations	Workforce execution
Financials	Payment & withdrawal

34.3 Operational Objectives

The lifecycle architecture supports:

- End-to-end workforce coordination
 - Scalable staffing operations
 - Real-world workforce execution
 - Financial operational consistency
-

34.4 Architecture Principle

Important architecture rule:

Operational execution begins after hiring lifecycle completion.

35. Canonical Source of Truth

CrewAnywhere2.0 architecture is governed by official canonical reference documents.

35.1 Official Canonical References

Primary architecture references:

1. CrewAnywhere2.0 Master Saved Memory & Database Architecture
2. CrewAnywhere2.0 Master Database Architecture Updated V3

3. CrewAnywhere2.0 Standardized Combined Lean FDR + Developer-Ready Technical FDR Standard
-

35.2 Canonical Governance Philosophy

Canonical references govern:

- Database architecture
 - Workflow systems
 - Lifecycle rules
 - Status transitions
 - Operational dependencies
 - Documentation standards
-

35.3 Architecture Governance Objectives

Canonical governance ensures:

- System consistency
 - Documentation alignment
 - Backend predictability
 - Operational scalability
-

35.4 Architecture Principle

Important architecture rule:

Canonical architecture references override conflicting legacy documentation.